ELSEVIER

Computer-Aided Design 31 (1999) 735–745

COMPUTER-AIDED DESIGN

www.elsevier.com/locate/cad

A pair-wise offset algorithm for 2D point-sequence curve

B.K. Choi*, S.C. Park

CAM Laboratory, Department of Industrial Engineering, Korea Advanced Institute of Science & Technology (KAIST), 373-1 Kusong-dong, Yusong-gu, Taejon, South Korea

Received 22 February 1999; received in revised form 5 August 1999; accepted 16 August 1999

Abstract

Presented in the paper is an efficient pair-wise offset algorithm for closed 2D *point-sequence curves* (PS-curve). A key feature of the proposed algorithm is that all *local invalid loops* are removed from the input PS-curve before constructing a raw offset-curve, by invoking a *pair-wise interference-detection* (PWID) test. In the PWID test, each pair of elementary offset segments is tested for interference and then interfering segments are successively removed. The proposed algorithm has been implemented and tested with various PS-curves. Empirical tests show that the proposed PS-curve offsetting method is very fast and robust with a near $O(n)$ time-complexity, where n is the number of points in a PS-curve. © 1999 Elsevier Science Ltd. All rights reserved.

Keywords: Offset curves; Polylines; Interference detection; Curve offsetting; Point-sequence curve; Pair-wise offset algorithm

1. Introduction

A dense sequence of 2D points representing a closed curve is called a (closed) *point-sequence curve* or *PS-curve* for short, which is a *polygon consisting mostly of short line segments*. PS-curve offsetting is an important operation in CAD/CAM, and the problem has widely been studied, mostly, as a *pocket-machining* problem [1–3]. In the literature, the PS-curve offsetting problem has been approached from three different directions: pair-wise offset [1], Voronoi diagram [2], and pixel-based approach [3].

The conventional *pair-wise offset approach* [1] consists of four steps: (1) an *offset segment* is generated for each element of *boundary PS-curve*; (2) a *raw offset-curve* is constructed by closing the gaps with trimming arcs; (3) all pair-wise self-intersections in the raw offset-curve are detected; and (4) all *invalid loops* are removed. The basic geometric entities related to the pair-wise offset method are shown in Fig. 1. The thick curve denotes a *boundary PS-curve* and the thin curve its *raw offset-curve*. A raw offset-curve is divided into *loops* at its *self-intersection points*, and a loop that has to be removed to obtain valid offset curves is called an *invalid loop*. As shown in Fig. 1, there are two types of invalid loops: *local invalid-loop* bounded by a single self-intersection point; *global invalid-loop* bounded by a pair of self-intersection points.

The pair-wise offset method is intuitive and simple in logic. As pointed out by Held et al. [2], however, the intersection-detection step (Step 3) is time consuming, and the loop-removal step (Step 4) is prone to numerical errors. The *Voronoi diagram method* [2,4] is known to be more efficient and robust, but it may also suffer from numerical instability [5]. The merit of the *pixel-based methods* [3,6] is robustness, but they would require a large amount of memory as well as an excessive computation time to achieve a desired level of precision.

In the mean time, a few algorithms have been proposed to cope with the above-mentioned difficulties of the conventional pair-wise offset approach. For example, algorithms for removing invalid loops are presented in Kalmanovich and Nisnevich [7] and in Rohmfeld [8], and an algorithm for detecting self-intersection points is presented in Park and Choi [9]. Even with the help from these new "remedies", the traditional pair-wise offset approach still suffers from numerical instability (to be elaborated later).

Presented in the paper is an algorithm for offsetting a closed PS-curve having no "islands" (Island PS-curves have to be manually bridged to the boundary PS-curve to make them a single closed PS-curve). One of the salient features of the pair-wise offset algorithm is that all *local invalid loops* are removed before constructing a raw offset-curve by invoking a *pair-wise interference-detection* (*PWID*) test. During the PWID test, each pair of elementary offset segments is tested for interference and then *interfering segments* are successively removed. The resulting raw offset-curve that now contains only *global invalid loops* (if they exist) is subjected to a sweep-line algorithm [9,10] to

* Corresponding author. Fax: +82-42-861-1692.
E-mail address: bkchoi@bezier.kaist.ac.kr (B.K. Choi)

0010-4485/99/$ - see front matter © 1999 Elsevier Science Ltd. All rights reserved.
PII: S0010-4485(99)00060-3

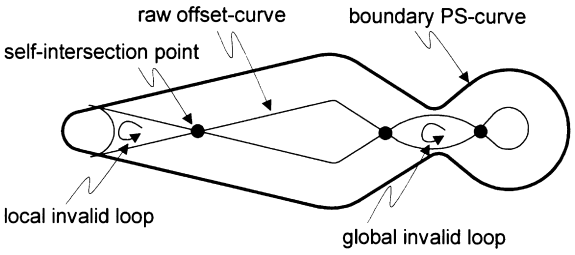


Fig. 1. Basic geometric entities in the pair-wise offset approach.

find all self-intersections. If global invalid loops are detected, they are removed by invoking the PWID test again. The overall procedure of the proposed *pair-wise offset algorithm* is as follows:

1. All data points belonging to *local invalid loops* are removed from the input *boundary PS-curve* by applying PWID tests.
2. A raw offset-curve is constructed from the resulting boundary PS-curve.
3. All the pair-wise self-intersections in the raw offset-curve are detected.
4. All *global invalid loops* are removed from the PS-curve by applying PWID tests.
5. Valid offset-curves are obtained from the final PS-curve.

The overall structure of the paper is as follows. The next section presents a motivation together with preliminary definitions. The PWID test, which is the heart of our offset algorithm, is described in detail in the section that follows. The pair-wise offset algorithm is given in Section 4, followed a section on illustrative examples and performance tests. The final section contains concluding remarks.

2. Motivation and preliminary definitions

A point-sequence curve (PS-curve) obtained by intersecting a sculptured surface with a plane may consist of thousands of densely spaced points or short line segments. Shown in Fig. 2(a) are an input *boundary PS-curve* and its *raw offset-curve* (the inner rectangle). A "corner" of the inner rectangle is in fact a *multiple-intersection area* in which a large number of *offset-segments* (i.e. offset line-segments) are intersecting with each other. A (100 times) magnified view of a multiple intersection area is shown in Fig. 2(b), which is magnified further (another 100 times) in Fig. 2(c).

The high concentration of multiple intersections (see Fig. 2(c)) at the multiple-intersection area of the raw offset-curve comes from a "near circular" portion of the boundary PS-curve, which is a major source of the numerical instability inherent in the conventional pair-wise offset method. This numerical instability stemming from a near-circular portion of the boundary PS-curve is called a *near-circular singularity*.

As pointed out by Sugihara [5], the Voronoi diagram method [2] may also suffer from a similar singularity problem if it is applied to a boundary PS-curve containing near circular portions. This point is clearly illustrated in Fig. 3. Shown in Fig. 3(a) are *bisector-lines* of the same boundary PS-curve (Note that the four sides are straight lines). Zoomed up views of the multiply intersecting bisector-lines at a corner point are shown in Fig. 3(b) and Fig. 3(c). When constructing Voronoi diagrams, these multiply intersecting bisector-lines may have to be trimmed off in some way.

If the *offset distance* in Fig. 2(a) is increased further, the raw offset-curve will have four well-defined *local invalid loops* that may easily be removed. However, as the offset distance is decreased back to the "radius" of a near circular portion of the boundary PS-curve, a large number of *degenerate local loops* may be formed at the multiple-intersection area of the raw offset curve. Thus, one may expect that the near circular singularity problem may be avoided if a raw offset-curve having no degenerate local loops can be constructed. Our pair-wise offset algorithm is based on this premise.

Now we introduce some preliminary definitions in order to provide a systematic description of the proposed pair-wise

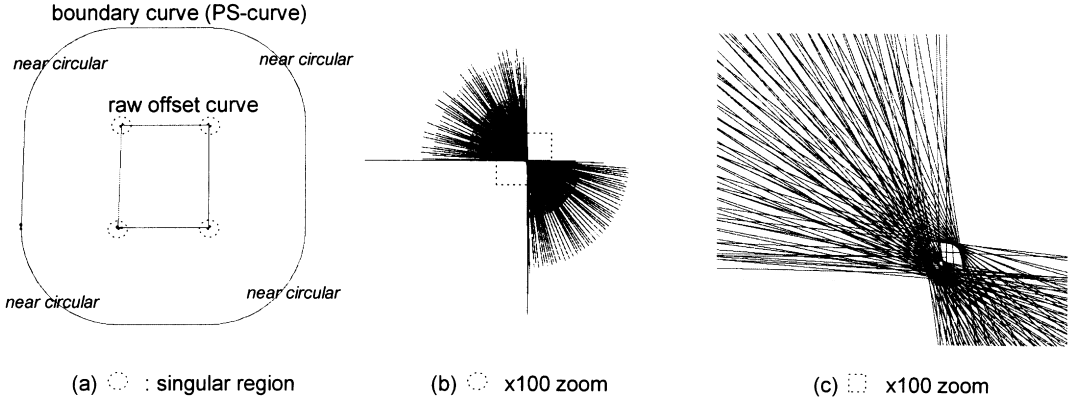


Fig. 2. Near circular singularity in pair-wise offsetting.

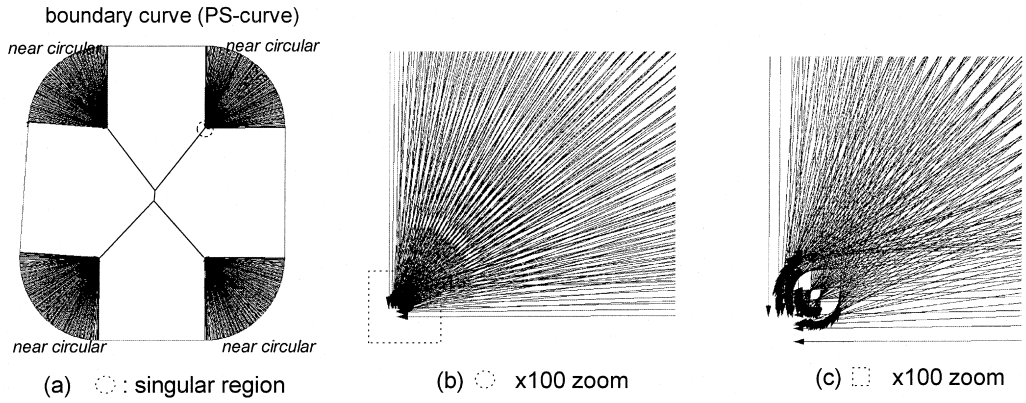


Fig. 3. Near circular singularity in Voronoi diagram construction.

offset algorithm. Noting that a closed PS-curve is a *polygon*, the term *vertex* is used to denote a point in the PS-curve and *edge* is used for a line segment joining two consecutive *vertices*.

Definition 1 (Vertex type [11]). For an inward (outward) offset, a vertex v of a closed PS-curve is called a *reflex vertex* if the internal (external) angle between the two *edges* incident upon v is greater than π and is called a *convex vertex* if the angle is less than π. Otherwise, the vertex is called a *collinear vertex*.

Definition 2 (Segment type). An *edge* (i.e. line segment joining two vertices) in a PS-curve is defined as a *linear segment*, and a *reflex vertex* is defined as a *reflex segment*.

In the proposed pair-wise offset algorithm, a (closed) *PS-curve is regarded as a parametric curve consisting of a closed sequence of linear/reflex segments*. A parameter range of 1 is assigned to each *segment* of the PS-curve. For example, the PS-curve in Fig. 4 consists of eight segments (seven linear segments and one reflex segment), and its parameter range is defined over $t \in [0,8]$. The *position* of the i-th segment at $t \in [i-1, i]$ is denoted by $\mathbf{P}(t)$ and the *unit normal* at the position by $\mathbf{N}(t)$ whose direction is the same as the *offset direction*. By definition, the position of a reflex segment is a single point, but its unit normal changes from the unit normal of the previous linear-segment to that of the next linear-segment. In Fig. 4, for example, **a** and **b** are unit normals of the second and fourth segments, respectively. Thus, the unit normal $\mathbf{N}(t)$ for $t \in [2,3]$ of the reflex segment changes from **a** (at $t = 2$) to **b** (at $t = 3$).

Definition 3 (Tangential circle). A circle of radius ρ making a tangential contact with a *segment* at a position $\mathbf{P}(t)$ is called its *tangential circle*, and the tangential circle at the end of the segment (i.e. at the *end-point*) is called an *end tangential-circle*. By definition, the *center point* $\mathbf{C}(t)$ of the tangential circle at t is given by

$$\mathbf{C}(t) = \mathbf{P}(t) + \rho \cdot \mathbf{N}(t),$$

where ρ is the offset distance and $\mathbf{N}(t)$ is the unit normal at $\mathbf{P}(t)$. A tangential circle in contact with more than one segment is called a *common tangential-circle*.

Definition 4 (Interfering point/range). A point t in the parameter domain of a PS-curve becomes an *interfering point* if its *tangential circle* intersects with the PS-curve. A range of interfering points is called an *interfering range*. An interfering range corresponding to a *local invalid-loop* (see Fig. 1) is called a *local interfering-range* (*LIR*) and an interfering range belonging to a *global invalid-loop* is called a *global interfering-range* (*GIR*).

Shown in Fig. 5 are a boundary PS-curve consisting of 26 segments (18 linear, eight reflex), three *common tangential-circles*, and two *interfering ranges*. In the figure, the range of the PS-curve is [0,26] and the interfering ranges are given

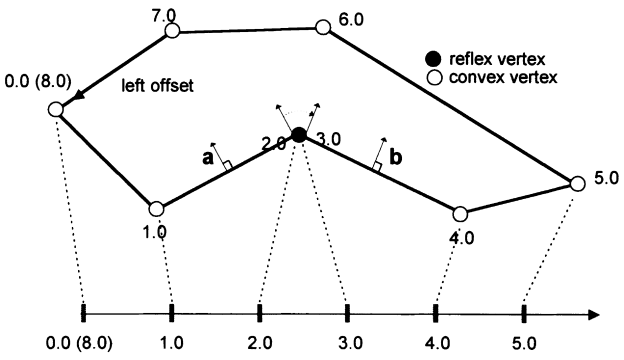


Fig. 4. Parameterization of a PS-curve.

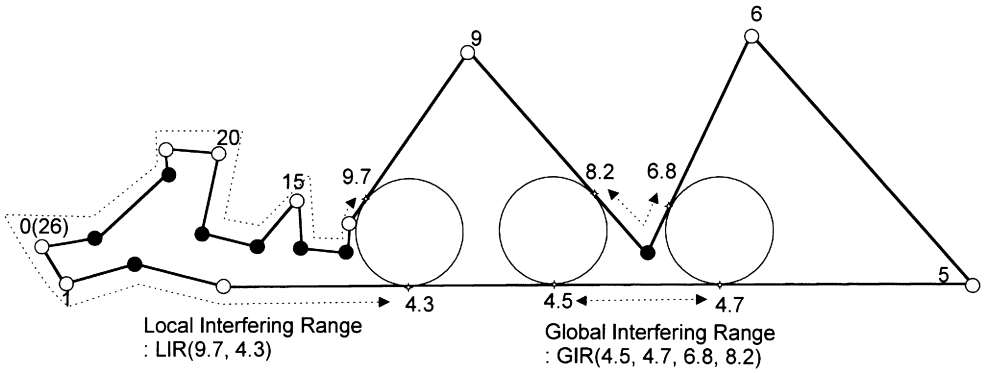


Fig. 5. Local and global interfering ranges in a PS-curve.

by

$$\mathrm{LIR} = [9.7, 4.3] \equiv [9.7, 26] \cup [0, 4.3];$$

$$\mathrm{GIR} = [4.5, 4.7] \cup [6.8, 8.2].$$

A GIR is bounded by two common tangential-circles; an LIR is bounded by one common tangential-circle. Since a *convex vertex* is an interfering point, any of them (hollow dots in Fig. 5) may be selected as a *seed point* for detecting a local interfering range. A solid dot denotes a *reflex vertex* (i.e. *reflex segment*); there are no collinear vertices in Fig. 5.

Starting from a *convex vertex* at $t = j$ as a *seed point*, the PWID-test works as follows. First, the segment beginning at $t = j$ is designated as a *forward segment* (*f_seg*) and the one ending at $t = j$ as a *backward segment* (*b_seg*). Then, *tracing directions* are defined so that the seed point becomes the *common start-point* of *f_seg* and *b_seg*. Thus, the *end-points* of *f_seg* and *b_seg* are located at $t = j + 1$ and $t = j - 1$, respectively. Now, if the end point of a segment is an *interfering point* (with respect to the other segment), this segment is replaced with the "next" segment along its tracing direction.

Definition 5 (Interfering segment). Let S1 and S2 be the two segments in a PWID test. Then, S1 becomes a *full-interfering segment* if its *end tangential circle* intersects with S2, *partial-interfering segment* if some of its tangential circles other than the end tangential-circle intersect with S2, and *reverse-interfering segment* if its tangential circles do not intersect with S2.

Definition 6 (Interfering relation). In a PWID test, we have three types of interfering relations between two segments S1 and S2. *Partial-interfering relation* if both are *partial interfering segments*; *reverse-interfering relation* if one or both of them are *reverse-interfering segments*; *full-interfering relation* otherwise (i.e. one is full-interfering segment, and the other is either partial- or full-interfering segment).

The two segments S1 and S2 in Fig. 6(a) are in a *partial-interfering relation* because both of them are partial interfering segments (i.e. a common tangential circle can be defined). The relation in Fig. 6(b) is a *full-interfering relation* because S1 is a full-interfering segment and S2 is a partial-interfering segment.

The two segments S1 and S2 in Fig. 7 are in a *reverse-interfering relation* because S1 is a reverse-interfering segment and S2 is a partial-interfering segment. The reverse-interfering relation in Fig. 7 signifies that all the reverse-interfering segments (with respect to S2) following S1 belong to the same *local interfering range* (LIR) which has to be bounded by a common tangential circle.

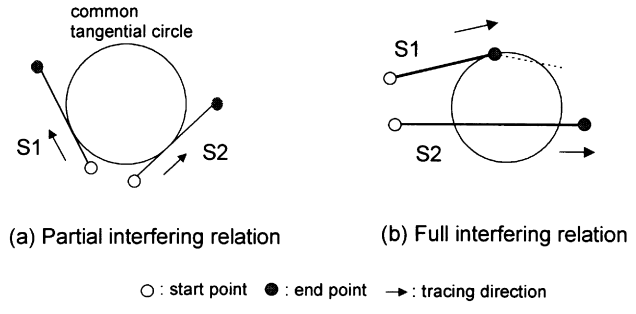


Fig. 6. Partial and full interfering relations.

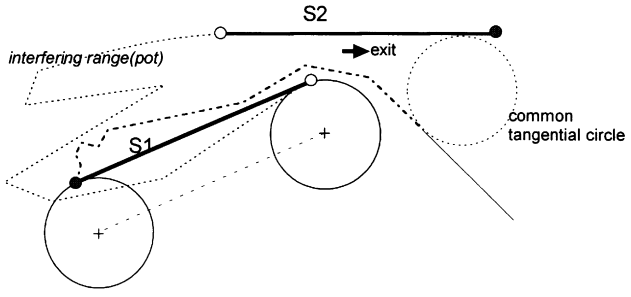


Fig. 7. Reverse interfering relation.

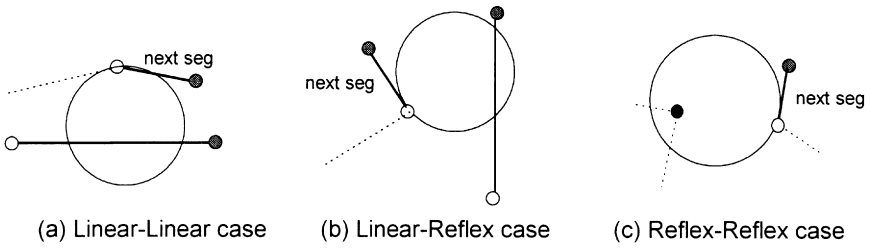


Fig. 8. Replacing a full-interfering segment.

3. Pair-wise interference detection test

We now present a PWID algorithm in which the following variables are employed:

- *f, b*: parameters of the boundary PS-curve for forward and backward tracing.
- *f_seg, b_seg*: forward segment and backward segment.

Input to the PWID-algorithm is a pair of starting parameter values (f_s, b_s). The first step is to get a *forward segment* (*f_seg*) starting at $f = f_s$ and a *backward segment* (*b_seg*) starting at $b = b_s$. Next, the *interfering relation* between *f_seg* and *b_seg* is identified. Then, one of the following two operations is repeated until *f_seg* and *b_seg* are in a *partial-interfering relation*:

- *Replace-full-segment*(*f_seg, b_seg*): If the two segments are in a *full-interfering relation*, a *full-interfering segment* is selected and it is replaced by its next segment in the PS-curve along the tracing-direction (*f_seg* or *b_seg* is updated accordingly). Shown in Fig. 8 are three cases of this operation, where the end tangential-circle of a full-interfering segment is denoted by a circle. In Fig. 8(b) and (c), a reflex-segment is being replaced by a linear-segment.
- *Replace-reverse-segment*(*f_seg, b_seg*): In case of *reverse-interfering relation*, the *reverse-interfering segment* is replaced by the first non-reverse-interfering segment along the tracing-direction skipping all the consecutive reverse-interfering segments (*f_seg* or *b_seg* is updated accordingly). Shown in Fig. 9 is the updated segment after this operation is applied to the reverse-interfering segment of Fig. 7.

When the PWID-test is terminated with a *partial-interfering relation*, the following operation is performed to return a pair of "end-of-interfering" values:

- *End-of-interfering*(*f_seg, b_seg, f, b*): In case of a *partial-interfering relation*, the common tangential circle of *f_seg* and *b_seg* is defined and the tangential contact points are computed. As shown in Fig. 10(a), the *tangential contact points*, $f = f_e$ and $b = b_e$, are obtained by intersecting the offset segments. Note in Fig. 10(b) and (c) that the offset segment of a reflex-segment is a circular arc.

However, it is possible for the two segments, *f_seg* and *b_seg*, to "bump" each other before the partial-interfering relation is encountered. This case arises only when the boundary PS-curve has no valid offset curves. Taking this case into account, the PWID process described above may be expressed as follows:

Pair-wise-interference-detection(*f, b*)
// *f*: forward-tracing parameter; *b*: backward-tracing parameter //
// *f_seg*: forward-tracing segment; *b_seg*: backward-tracing segment //
Begin
Get tracing segments *f_seg* and *b_seg* corresponding to *f* and *b*;
Repeat {
If *end-points* of *f_seg* and *b_seg* are equal then set $f = b$ and return(*f, b*);
Identify the interfering-relation of *f_seg* and *b_seg*;
If *Full-interfering-relation* then Replace-full-segment(*f_seg, b_seg*);
If Reverse-interfering-relation then Replace-reverse-segment(*f_seg, b_seg*);
} Until (Partial-interfering-relation);
End-of-interfering (*f_seg, b_seg, f, b*);
return(*f, b*);
End.

If the above PWID-function is called with the parameter values ($f = f_s$, $b = b_s$) at starting points of interfering regions, it will return the parameter values ($f = f_e$, $b = b_e$) at ending points of the interfering regions. Consequently, an interfering region (IR) is obtained as:

$$IR = [b_e, b_s] \cup [f_s, f_e].$$

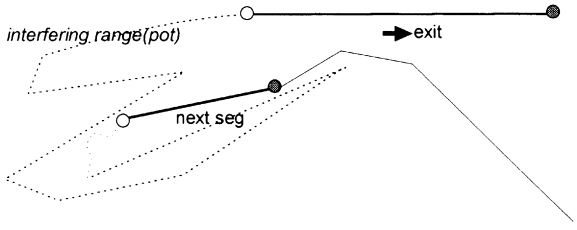


Fig. 9. Replacing all reverse-interfering segments.

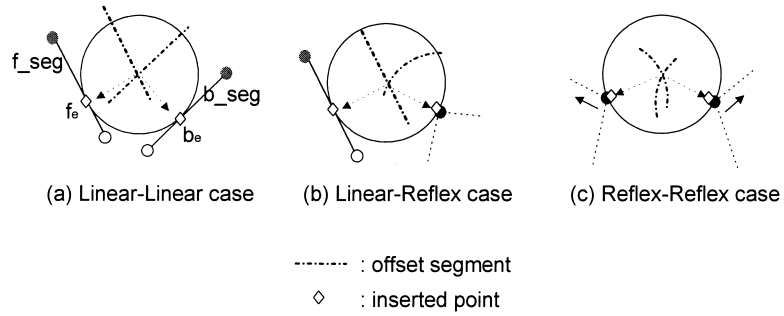


Fig. 10. End of tracing with a partial-interfering relation.

The above IR interval reduces to $[f_s, b_s]$ if $f_e = b_e$ and to $[b_e, f_e]$ if $f_s = b_s$. In general, the interval of an IR is defined as a modulus of "n" (the number of segments in a PS-curve). Namely, if $a > b$, the interval $[a, b]$ is to be interpreted as $[a, n] \cup [0, b]$. With the PWID-function on hand, we are now ready to present the new *pair-wise offset algorithm* for PS-curve offsetting.

4. Pair-wise offset algorithm and its time-complexity

As discussed earlier, in the *pair-wise offset algorithm* proposed in this paper, valid offset curves of an input *boundary PS-curve* are obtained in five steps:

1. All *local interfering ranges* (LIR) are removed from the boundary PS-curve.
2. A *raw offset-curve* is constructed from the remaining elements of the PS-curve.
3. All *pair-wise self-intersections* in the raw offset-curve are detected.
4. All *global interfering ranges* are removed from the raw offset-curve.
5. *Valid offset-curves* are obtained from the remaining portions of the raw offset-curve.

Initially, the PS-curve is given as a closed sequence of 2D points, where each point is regarded as a *vertex*. As a *preprocessing step*, collinear-vertices (Definition 1) may be removed from the initial PS-curve, which is not essential for the algorithm but may help speed up the subsequent processes. Then, the PS-curve is converted into an anti-clockwise linked-list of *segments* (Definition 2), during which a list of pointers (to the PS-curve) for the *convex vertices* is also constructed.

We present the *main algorithm* first, and then additional explanations are provided later. The proposed PS-curve offsetting algorithm is as follows:

Pair-Wise-Offset-Algorithm:
// Input: PS-curve, offset direction, offset distance (ρ).
//
1. Remove all LIR:
While (there remains a *convex-vertex* in the *PS-curve*) **do** {
(1) Get the parameter value t_c of a *convex-vertex*;

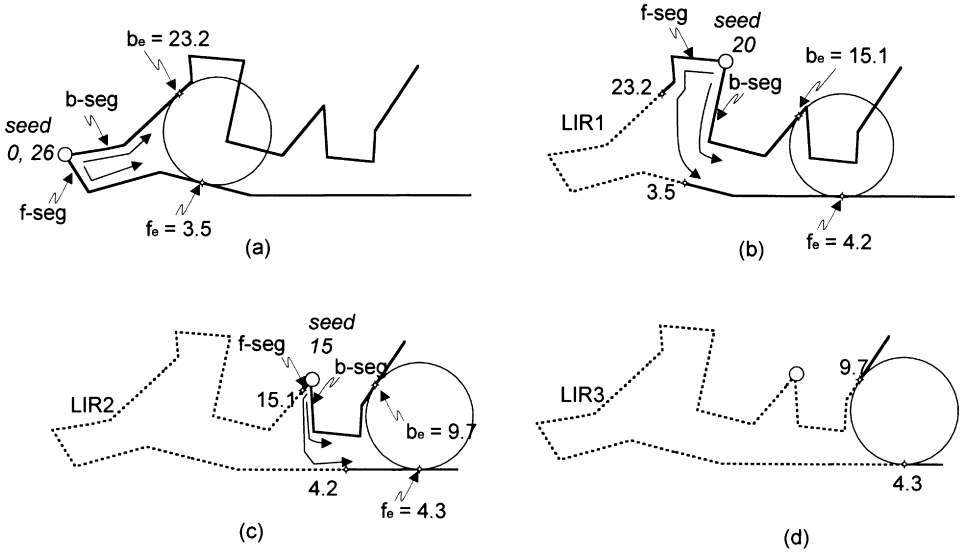


Fig. 11. Detection and deletion of LIR.

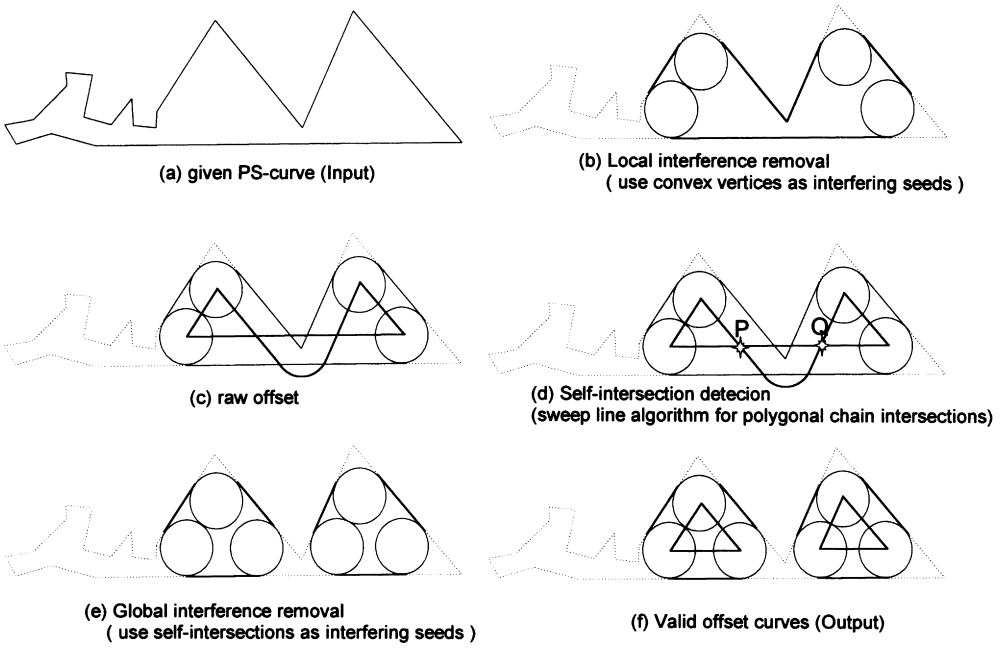


Fig. 12. Raw offset-curve and deletion of global interfering range.

(2) Set $f = t_c$ and $b = t_c$;
(3) Pair-wise-interference-detection(f, b);
(4) Remove the LIR $= [b, f]$ from the PS-curve;
}; // End-of-while //

2. Construct a *raw offset-curve* from the remaining segments in the PS-curve.
3. Find all *self-intersections* of the *raw offset-curve* using a sweep-line algorithm [9,10].
4. Remove all global interfering ranges (GIR):
   GIR $= \Phi$;
   While (there remains an "unmarked" *self-intersection* in the *raw offset-curve*) **do**
   {
   (1) Select an unmarked *self-intersection* point $P = (f_s, b_s)$ and mark it;
   (2) Set $f = f_s$ and $b = b_s$;
   (3) Pair-wise-interference-detection (f, b);
   (4) Mark the self-intersection point $Q = (f, b)$;
   (5) GIR = GIR $\cup$ $[f_s,f] \cup [b,b_s]$;
   } ; // End-of-while //
   (6) Remove GIR from the *raw offset-curve*;
5. Construct *valid offset-curves* from the remaining segments of the offset-curve.

Step 1 of the algorithm will be explained with the *local interfering range* (LIR) of the PS-curve of Fig. 5 which is shown again in Fig. 11. As depicted in Fig. 11(a), the convex vertex at $t_c = 0$ is selected as a *local seed* point in Step 1-1. In Step 1-3, the *Pair-wise-interference-detection*(f, b) function is called with $f = b = 0$, which will return the parameter values ($f = f_e = 3.5$, $b = b_e = 23.2$) at the contact-points of the *common tangential circle*. In Step 1-4, the $\text{LIR}_1 = [23.2,3.5]$ is deleted from the PS-curve as depicted in Fig. 11(b). In the next iteration of the While-loop, another convex vertex at $t_c = 20$ is selected in Step 1-1, and then, in Step 1-3, the PWID function is called with $f = b = 20$ and the parameter values ($f = f_e = 4.2$ and $b = b_e = 15.1$) for a new LIR are returned. Then, in Step 1-4, the new interfering range, $\text{LIR}_2 = [15.1, 4.2]$, is deleted (see Fig. 11(c)). During another iteration of the While loop, another interfering range, $\text{LIR}_3 = [9.7, 4.3]$, is detected and it is deleted (see Fig. 11(d)).

In the above example, one may observe that a *segment* is visited only once even though it may belong to more than one LIR (since $\text{LIR}_1 \subset \text{LIR}_2 \subset \text{LIR}_3$). Thus, Step 1 has a *linear time-complexity*. More precisely, the time complexity of Step 1-1 is $O(n_c)$ and that of Step 1-3 is $O(n_L)$, where n_c is number of convex vertices and n_L is number of segments belonging to the LIR of the PS-curve. Step 1-2 involves assignment operations and Step 1-4 is a simple linked-list operation. Thus, as a whole, the time complexity of Step 1 is $O(n_L)$ since $n_L > n_c$.

A comment on the choice of local seed points in Step 1 may be in order. For the PS-curve of Fig. 11, the convex vertices at $t = 0$, 20, 15 were selected "at random" as local seed points (Step 1-1 of the algorithm). In theory, the performance of Step 1 can be improved by employing an "optimal" strategy for selecting a local seed point. In practice, however, defining such a strategy is not easy, and finding a better seed point will require an additional computational burden. Tests showed that the choice of a selection strategy had no significant effect on the overall performance of the algorithm.

The same PS-curve of Fig. 5 is shown again in Fig. 12(a).

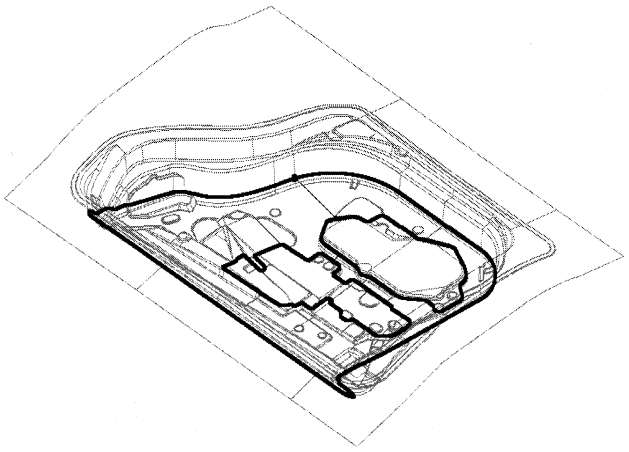


Fig. 13. PS-curves generated by plane/surface intersection.

Since all the segments belonging to LIR have been deleted from the PS-curve during Step 1, it now contains only five linear segments and one reflex segment (Fig. 12(b)). In Step 2 of the algorithm, a *raw offset-curve* of the resulting PS-curve is constructed (Fig. 12(c)). Thus, the time complexity of Step 2 is $O(n_R)$, where n_R is the number of segments in the *raw offset-curve which is free of any local invalid loops*.

In Step 3 of the algorithm, self-intersection points of the raw offset-curve are computed (Fig. 12(d)). If a sweep-line algorithm [9,10] is used in finding self-intersection points, it can be shown [9] that the time complexity of Step 3 is $O((n_R + s)\log m)$, where s is the number of self-intersections and m is the number of monotone chains in the raw offset-curve. For the example given in Fig. 12, we have $r = 6$, $s = 2$, and $m = 2$.

Now we describe detailed operations performed in Step 4 of the algorithm. In Step 4-1, a self-intersection point P (Fig. 12(d)) is selected and *tracing directions* are assigned by setting $f_s = 4.5$ and $b_s = 8.2$ (see Fig. 5). Since the self-intersection point P of the raw offset-curve is the center point of a tangential circle of the PS-curve (Fig. 5), the tracing directions are assigned such that the tangential circle "gouges" the PS-curve. Now using the selected point P($f_s = 4.5$, $b_s = 8.2$) as a *global seed* point, the *Pair-wise-interference-detection*(f, b) function is called with $f = 4.5$ and $b = 8.2$ in Step 4-3. The PWID function will then return the parameter values at another self-intersection point $Q(f = 4.7,\ b = 6.8)$. The two self-intersection points P, Q are "marked" in Step 4-1 and Step 4-4, respectively. In Step 4-5, the *global interfering range*, GIR $= [4.5, 4.7] \cup [6.8, 8.2]$, detected in Step 4-3 is collected. The collected GIRs are deleted *after* the While-loop (i.e. in Step 4-6) because it may be possible for small portions of some GIRs being traced more than once.

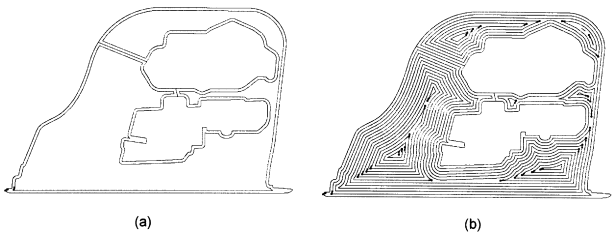


Fig. 14. Offsetting of the door PS-curve: (a) singe offset; (b) full offset.

The time complexity of Step 4-1 and Step 4-4 combined is $O(s)$ where s is the number of self-intersections in the raw offset-curve and that of Step 4-3 is about $O(n_G)$ where n_G is the number of segments belonging to global interfering ranges (the actual time complexity of Step 4-3 may be slightly higher due to the possibility that small portions of some GIRs will be traced more than once). Since $s \ll n_G$, the time complexity of Step 4 is $O(n_G)$. Finally, the time complexity of Step 5 is $O(n_V)$, where n_V is the number of segments in the valid offset-curves.

In summary, let $O_j(\ )$ denote the time-complexity of Step j of the pair-wise offset algorithm, then the overall time-complexity of the algorithm may be expressed as:

$$O_1(n_L) + O_2(n_R) + O_3((n_R + s)\log m) + O_4(n_G) + O_5(n_V),$$

where n_L: number of segments belonging to LIR in a PS-curve, n_R: number of segments in the raw offset-curve,

n_G: number of segments belonging to global interfering ranges,
n_V: number of segments in valid offset-curves,
s: number of self-intersections in the raw offset-curve, and
m: number of monotone chains in the raw offset-curve.

It should be noted that the *raw offset-curve* is free of any local invalid loops. Each step of the algorithm has a linear time-complexity except Step 3. However, in practical applications where input PS-curves contain a large proportion of interfering segments, majority of the time would be used up in Step 1 (and Step 4), which will make the time-complexity of the algorithm linear, as will be seen in the next section.

5. Empirical performance tests

The proposed algorithm has been implemented[1] and tested with various types of PS-curves. Shown in Fig. 13 is a CAD model of an automobile-part (door panel) stamping-die. The PS-curves (the thick curves in the figure) were obtained by intersecting the die surface with a horizontal plane. Since the proposed algorithm works on a closed PS-curve (or a polygon), the two "island" PS-curves have to be (manually) bridged to the main PS-curve, which is required when generating pocket-machining cutter-paths. Shown in Fig. 14(a) is an inward *single-offset* curve (as well as the "door" PS-curve) generated by the proposed offsetting algorithm with an *offset distance* (ρ) of 10 mm. If the offsetting

[1] Interested readers are welcome to get a copy of the pair-wise offset program, as well as sample data including the ones discussed in the paper from, http://bezier.kaist.ac.kr/~camlab/demosw.html.

Table 1
Execution times (ms) for the door ps-curve offset

Number of points	Tolerance	Single offset	Full offset (Step 1%)
3877	0.001	140	290 (90%)
3266	0.002	130	270 (93%)
2877	0.003	130	270 (89%)
2605	0.004	120	260 (92%)
2383	0.005	90	240 (83%)
2201	0.006	90	230 (87%)
2022	0.007	80	230 (83%)
1852	0.008	70	200 (80%)
1695	0.009	70	190 (84%)
1535	0.01	60	170 (76%)
858	0.02	50	130 (69%)
574	0.03	20	80 (75%)
436	0.04	20	40 (75%)
360	0.05	20	40 (50%)

algorithm is repeatedly invoked with the same ρ, *full-offset* curves are generated as depicted in Fig. 14(b).

As discussed earlier, the *number of data points* (n) in a PS-curve becomes larger as the *intersection tolerance* (τ) becomes smaller. In the case of the door PS-curve of Fig. 13, $n = 3877$ when $\tau = 0.001$ mm (which is a typical tolerance value for finish machining); $n = 360$ when $\tau = 0.05$ mm. To examine the computational behavior of the pair-wise offset algorithm, 14 cases of the PS-curve in Fig. 13 were generated using different values of τ ranging from 0.001 to 0.05. Summarized in Table 1 are execution times for the single-offset (Fig. 14(a)) and full-offset (Fig. 14(b)) for each case of the door PS-curve. Also indicated in the "Full offset" column are percentages of time taken up in Step 1 of the algorithm. The execution times (in mil-seconds) in Table 1 were obtained by executing the program on a PC with an MMX200 Processor running under Linux.

Instead of the "theoretical" time-complexity analysis

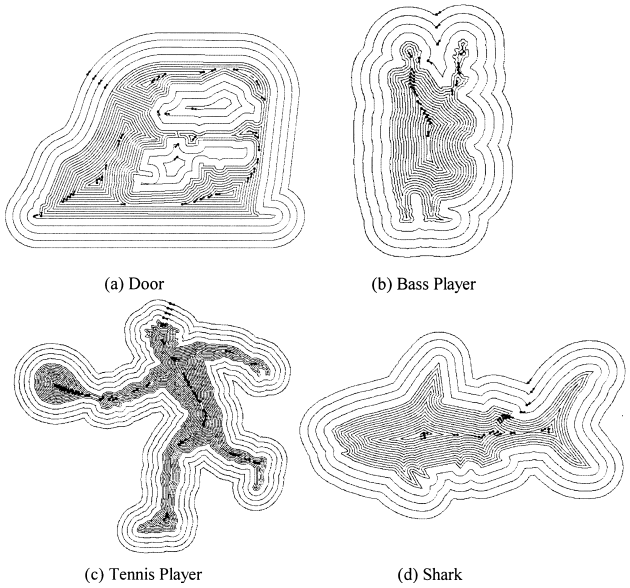


Fig. 15. PS-curve offsetting examples.

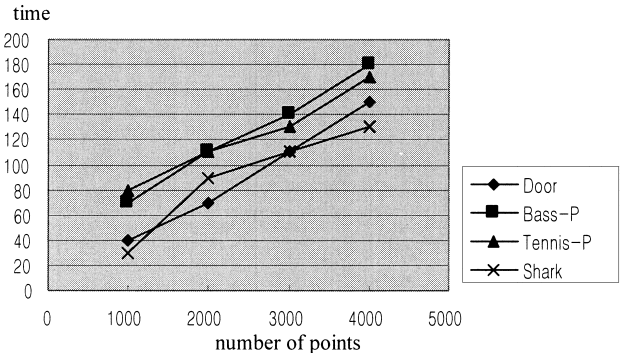


Fig. 16. Single offset times for the example curves in Fig. 15.

given in the previous section, practitioners would be more interested in the performance of the algorithm in terms of following factors. The number of data points (n); the number of *bottlenecks* (*vortexes*) in the PS-curve (β); offset-distance (ρ); the effect of repeated offsetting (*single*/*full*) or offset-directions (*inward*/*outward*). For this purpose, the offsetting program was tested on four different PS-curve shapes, including the "door-curve" of Fig. 13.

Shown in Fig. 15 are four types of PS-curves together with their (inward and outward) offset curves. They are termed (a) Door, (b) Bass player, (c) Tennis player, and (d) Shark. The number of bottlenecks in each PS-curve shape is as follows:

$$\beta_{\text{door}} = 8; \quad \beta_{\text{bass}} = 10; \quad \beta_{\text{tennis}} = 15; \quad \beta_{\text{shark}} = 2.$$

Four PS-curves were generated for each PS-curve shape, with the number of data points n ranging from 1000 to 4000 with an increment of 1000. Thus, in all we have 16 PS-curves that are to be used in evaluating our offsetting code (The program was executed on an IBM Workstation RS6000).

A *single offset* (inward) with an offset-distance of 10 ($\rho = 10$) was computed for each of the 16 PS-curves, and the result is plotted in Fig. 16. One may observe from the graphs that the offsetting algorithm exhibits a *linear* time-complexity with respect to the number of data points (n). The same trend is true for the execution times given in

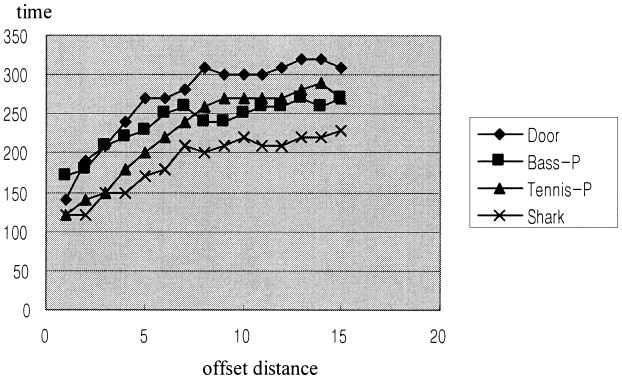


Fig. 17. Effect of offset distance on the single offset time.

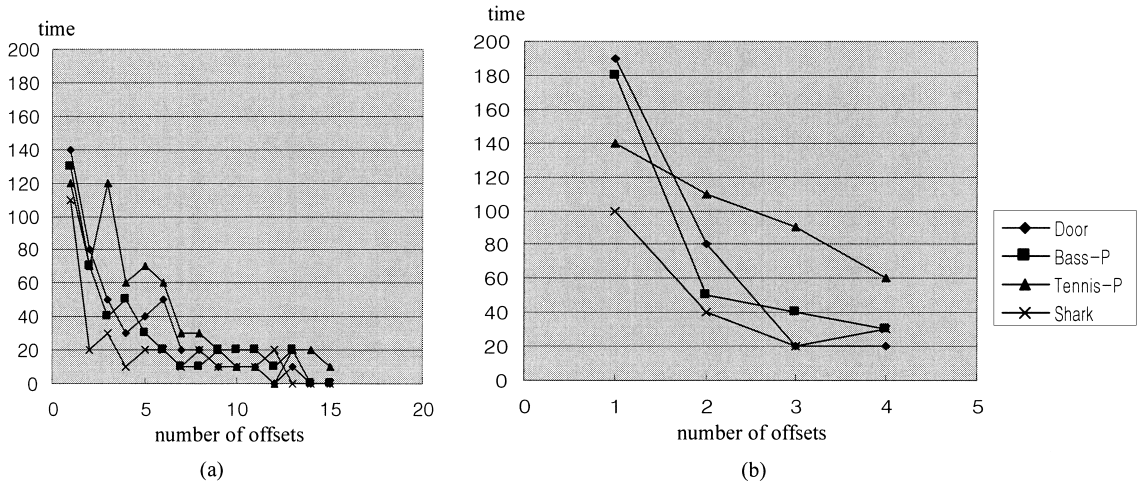


Fig. 18. Effect of repeated offsetting: (a) inward; (b) outward.

Table 1 (for both single and full offsets). Also, there seems to be a positive correlation between the execution time and the number of bottlenecks (β); the execution times for the Bass-player curve and the Tennis-player curve ($\beta_{bass} = 10$; $\beta_{tennis} = 15$) are higher than those for the Door curve and the Shark curve ($\beta_{door} = 8$; $\beta_{shark} = 2$).

To examine the effect of offset radius (ρ), a series of single offset-curves was computed by increasing ρ by one-fifteenth of the *full inward-offset distance* for each PS-curve shape (with $n = 4000$). The result is plotted in Fig. 17, which shows that its execution time increases almost linearly with respect to ρ up to the half of the full offset distance and levels off after that point.

Finally, to examine the effect of *repeated offsetting* (an offset curve is used as an "input PS-curve" in a subsequent offsetting operation), each PS-curve shape with $n = 4000$ was offset inward 15 times and offset outward four times (See Fig. 15), and their results are plotted in Fig. 18. In both inward and outward offsets, the execution time for a single-offset decreases rather sharply as the offsetting operation is repeated. This trend is due to that fact that an offset PS-curve becomes much simpler (i.e., has fewer *local interfering points*) than its input PS-curve.

6. Discussion and conclusions

Presented in this paper is an efficient pair-wise offset algorithm in which the *invalid loop removal* problem is reduced to a line/line or line/circle intersection problem. In particular, by deleting all the local interfering segments in the input PS-curve *before* constructing its raw offset-curve, the *near circular singularity* is avoided. Further improvement in Step 3 (finding self-intersections in the raw offset-curve) of the algorithm comes from the use of the *sweep-line algorithm* [9], which is a modification of the well-known Bentley–Ottmann algorithm [10].

Empirical tests showed that the time-complexity of the algorithm is almost linear with respect to the number of points in the input PS-curve. For the example PS-curves given in Fig. 15, the proposed algorithm performed favorably over a couple of Voronoi codes including Lark [2]. But, this does not necessarily mean that the proposed method is better than Voronoi-based offsetting in general as Voronoi methods are being improved [12–14]. The authors tried unsuccessfully to obtain "state-of-the-art" Voronoi codes for benchmarking. Instead, as announced in Section 5, a working code of the proposed algorithm will be made available for public benchmarking.

The main drawback of the proposed algorithm is that it is applicable only to a PS-curve (consisting of linear segments) having no "islands". Thus, a further research is required to handle circular-arc segments and islands (At present, a circular arc is to be converted to a dense sequence of points, and islands have to bridged as shown in Fig. 14). Another need for improvement is in Step 4, where *global interfering-ranges* may be detected and removed by using the loop-removal method proposed in Rohmfeld [8].

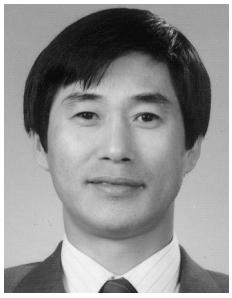
Acknowledgements

The authors wish to thank Dr Tamas Varady of Hungarian Academy of Sciences for making the offsetting package Lark available for benchmarking.

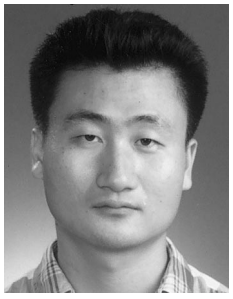
References

[1] Hansen A, Arbab F. An algorithm for generating NC tool path for arbitrary shaped pockets with islands. ACM Transactions on Graphics 1992;11(2):152–82.
[2] Held M, Lukacs G, Andor L. Pocket machining based on contour-parallel tool paths generated by means of proximity maps. Computer Aided Design 1994;26(3):189–203.
[3] Choi BK, Kim BH. Die-cavity pocketing via cutting simulation. Computer Aided Design 1997;29(12):837–46.

[4] Persson H. NC machining of arbitrary shaped pockets. Computer Aided Design 1978;10(3):169–74.
[5] Kokichi Sugihara. Degeneracy and Instability in Geometric Computation. Proceedings of IFIP WG5.2 GEO-6 Conference in Tokyo University, December 7–9. 1998. p. 5–15.
[6] Chiang CS, Hoffman CM, Lynch RE. How to compute offsets without self-intersection. SPIE Conference Proceedings Curves and Surfaces in Computer Vision and Graphics II, Boston, MA. 1991. p. 76–87.
[7] Kalmanovich G, Nisnevich G. Swift and stable polygon growth and broken line offset. Computer Aided Design 1998;30(11):847–52.
[8] Rohmfeld RF. IGB-offset for plane curves-loop removal by scanning of interval sequences. Computer Aided Geometric Design 1998;15:339–75.
[9] Park SC, Shin H, Choi BK. A sweep line algorithm for polygonal chain intersection and its applications. Proceedings of IFIP WG5.2 GEO-6 Conference in Tokyo University, December 7–9. 1998. p. 187–95 (also available from bkchoi@bezier.kaist.ac.kr).
[10] Bentley JL, Ottmann TA. Algorithms for reporting and counting geometric intersections. IEEE Transactions on Computers 1979;28:643–7.
[11] Held M. On the computational geometry of pocket machining, Berlin: Springer, 1991.
[12] Held M. Voronoi diagrams and offset curves of curvilinear polygons. Computer Aided Design 1998;30(4):287–300.
[13] Kim D-S. Polygon offsetting using a Voronoi diagram and two stacks. Computer Aided Design 1998;30(14):1069–76.
[14] Ramamurthy R, Farouki RT. Topological and computational issues in Voronoi diagram and medial axis constructions for planar domains with curved boundaries, Geometric Modeling and Processing '98BOOKPLUS Press, 1998. p. 1–26 ISBN 89-86518-10-0.



Byoung K. Choi is a professor of manufacturing systems engineering in the Department of Industrial Engineering at KAIST, which he joined in 1983. He received a BS from Seoul National University, an MS from KAIST, and a PhD from Purdue University, all in Industrial Engineering. His research interests are in the area of sculptured-surface modeling and machining, CAPP for die-cavity machining, discrete event system modeling and simulation, and virtual manufacturing system. He can be reached via email at bkchoi@bezier.kaist.ac.kr



Sang C. Park is a PhD student in the Department of Industrial Engineering at KAIST. He received his BS and MS degrees from KAIST in 1994 and 1996, respectively. His research interests include geometric algorithms in CAD/CAM, sculptured surface NC machining, and system modeling and simulation. He can be reached via email at *psc@gregory.kaist.ac.kr*